

Unidad I: Computabilidad

Reducciones e indecidibilidad

Clase 08 - Lógica para Ciencia de la Computación/Teoría de la Computación

Prof. Miguel Romero

El problema de la parada (Halting problem)

Definamos el siguiente lenguaje sobre $\{0, 1\}$:

$\text{Halting} = \{\langle M, w \rangle \mid M \text{ es una MT, } w \in \{0, 1\}^* \text{ y } M \text{ se detiene con } w\}.$

Proposición:

Halting es recursivamente enumerable.

Basta modificar la MUT U que presentamos hace algunas clases.

MUT modificada para Halting

Sobre entrada $z \in \{0,1\}^*$:

1. Verificar que $z = \langle M, w \rangle$ para alguna MT M y $w \in \{0,1\}^*$. Si no, **rechazar**.
2. Escribir w en la cinta 2 y poner la cabeza al inicio de w .
3. Escribir 0 en la cinta 3 (estado inicial).
4. Si el contenido de la cinta 3 es 0^n , **aceptar**. (n es la cantidad de estados.)
5. Buscar en la cinta 1 una transición de la forma $\delta(s_i, a) = (s_j, b, D)$ donde 0^i es el contenido de la cinta 3 y la cabeza de la cinta 2 está sobre a . Si no hay tal transición, **aceptar**.
6. Reemplazar 0^i por 0^j en la cinta 3.
7. Reemplazar a por b en la cinta 2.
8. Mover la cabeza de la cinta 2 a la izquierda o derecha, según D .
9. Volver al paso (4).

El problema de la parada (Halting problem)

Definamos el siguiente lenguaje sobre $\{0, 1\}$:

$\text{Halting} = \{\langle M, w \rangle \mid M \text{ es una MT, } w \in \{0, 1\}^* \text{ y } M \text{ se detiene con } w\}.$

Teorema:

Halting es indecidible.

Halting es indecible: demostración

Teorema:

Halting es indecible.

Por contradicción, supongamos que existe una MT H que decide Halting.

- H se detiene frente a toda entrada.
- Para toda MT M y $w \in \{0,1\}^*$:
 - H acepta $\langle M, w \rangle \implies M$ se detiene con w .
 - H no acepta $\langle M, w \rangle \implies M$ no se detiene con w .

Halting es indecidible: demostración

A partir de H , nos gustaría definir una MT A que decida **Accept**.

Como **Accept** es indecidible, esto sería una **contradicción**.

- No puede existir una MT H que decida Halting.

¿Cómo definimos la MT A a partir de la MT H ?

Halting es indecidible: demostración

Para toda MT codificable M , existe una MT codificable S_M tal que:

Para toda entrada $w \in \{0,1\}^*$:

M acepta $w \iff S_M$ se detiene con w .

Pseudocódigo de S_M

Sobre entrada $w \in \{0,1\}^*$:

1. Ejecutar M sobre w .
2. Si M **acepta** w , **aceptar**.
3. Si M **rechaza** w , quedarse en **loop infinito**.

¿Cómo implementaría la MT S_M ?

Halting es indecidible: demostración

Para toda MT codificable M , existe una MT codificable S_M tal que:

Para toda entrada $w \in \{0,1\}^*$:

M acepta $w \iff S_M$ se detiene con w .

Pseudocódigo de S_M

Sobre entrada $w \in \{0,1\}^*$:

1. Ejecutar M sobre w .
2. Si M **acepta** w , **aceptar**.
3. Si M **rechaza** w , quedarse en **loop infinito**.

OJO: $\langle S_M \rangle$ se puede computar a partir de $\langle M \rangle$.

Halting es indecidible: demostración

A partir de H , nos gustaría definir una MT A que decida **Accept**.

Pseudocódigo de A

Sobre entrada $\langle M, w \rangle$:

1. Ejecutar H sobre $\langle S_M, w \rangle$.
2. Si H **acepta**, **aceptar**.
3. Si H **rechaza**, **rechazar**.

Tenemos que:

$$\begin{aligned} A \text{ acepta } \langle M, w \rangle &\iff H \text{ acepta } \langle S_M, w \rangle \\ &\iff S_M \text{ se detiene con } w \iff M \text{ acepta } w. \end{aligned}$$

En otras palabras, $L(A) = \text{Accept}$.

Como A se detiene frente a toda entrada, A decide **Accept**.

Contradicción!

El problema $\text{Halting}_\varepsilon$

Definamos el siguiente lenguaje sobre $\{0, 1\}$:

$$\text{Halting}_\varepsilon = \{ \langle M \rangle \mid M \text{ es una MT y } M \text{ se detiene con } \varepsilon \}.$$

Proposición:

$\text{Halting}_\varepsilon$ es recursivamente enumerable.

Teorema:

$\text{Halting}_\varepsilon$ es indecidible.

Paréntesis: $\text{Halting}_\varepsilon$ vs Halting

Para toda MT codificable M y palabra $w \in \{0, 1\}^*$,
existe una MT codificable $B_{M,w}$ tal que:

M se detiene con $w \iff B_{M,w}$ se detiene con ε .

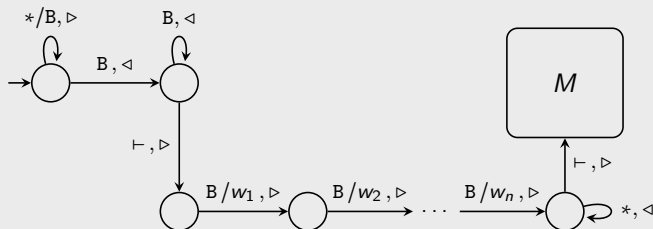
Pseudocódigo de $B_{M,w}$

Sobre entrada $x \in \{0, 1\}^*$:

1. Borrar x y dejar w escrito en la cinta.
2. Ejecutar M sobre w .

Paréntesis: $\text{Halting}_\varepsilon$ vs Halting

Implementación de $B_{M,w}$ (asumimos que $w = w_1 \cdots w_n$)



OJO: $\langle B_{M,w} \rangle$ se puede computar a partir de $\langle M, w \rangle$.

Halting _{ε} es indecidible: demostración

Por contradicción, supongamos que existe una MT K que decide Halting _{ε} .

A partir de K , definimos la siguiente MT H .

Pseudocódigo de H

Sobre entrada $\langle M, w \rangle$:

1. Ejecutar K sobre $\langle B_{M,w} \rangle$.
2. Si K **acepta**, **aceptar**.
3. Si K **rechaza**, **rechazar**.

Tenemos que:

$$\begin{aligned} H \text{ acepta } \langle M, w \rangle &\iff K \text{ acepta } \langle B_{M,w} \rangle \\ &\iff \langle B_{M,w} \rangle \text{ se detiene con } \varepsilon \iff M \text{ se detiene con } w. \end{aligned}$$

En otras palabras, $L(H) = \text{Halting}$.

Como H se detiene frente a toda entrada, H decide Halting. **Contradicción!**

Complementos de Halting y Halting_ϵ

Corolario:

$\overline{\text{Halting}}$ y $\overline{\text{Halting}_\epsilon}$ no son recursivamente enumerables.

¿Cómo demostramos esto?

Problema A:

Instalar la lámpara del living



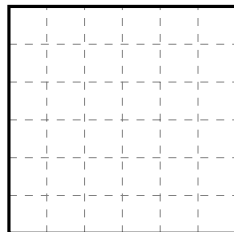
Problema B:

Encontrar un maestro



Problema A:

Calculo del área de un cuadrado



Problema B:

Multiplicación de dos números

base $\times$ altura

Reducciones: idea

Dados: problemas de decisión **A** y **B**.

Reducción de **A** a **B**:

- Transformar entradas de **A** a entradas de **B**.
- Si puedo resolver **B**, entonces puedo resolver **A**.

Notación: **A** se reduce a **B**.

Reducciones: ejemplo

Reducción de Accept a Halting:

- Transformar entradas de Accept a entradas de Halting.
- Si puedo resolver Halting, entonces puedo resolver Accept.

Accept

$\langle M, w \rangle$

$\mapsto$

Halting

$\langle S_M, w \rangle$

Propiedad clave que se debe cumplir:

$$\langle M, w \rangle \in \text{Accept} \iff \langle S_M, w \rangle \in \text{Halting}$$

Funciones computables

Definición:

Sean alfabetos finitos Σ_1 y Σ_2 . Sea una función $f : \Sigma_1^* \rightarrow \Sigma_2^*$ y una MT $M = (Q, \Sigma_1, \Gamma, q_0, q_f, \delta)$. Decimos que M **computa** f si:

1. Para toda entrada $w \in \Sigma_1^*$, la **ejecución** $\rho = C_0, \dots, C_m$ de M sobre w es **finita**. (M se detiene con w .)
2. $C_m = \vdash q_f f(w)$.

En otras palabras:

Para toda entrada $w \in \Sigma_1^*$, la MT M acepta, deja la cabeza en la posición 1, y deja escrito en la cinta la palabra $f(w) \in \Sigma_2^*$.

Definición:

Sean alfabetos finitos Σ_1 y Σ_2 , y $f : \Sigma_1^* \rightarrow \Sigma_2^*$ una función. Decimos que f es **computable** si existe una MT M que computa f .

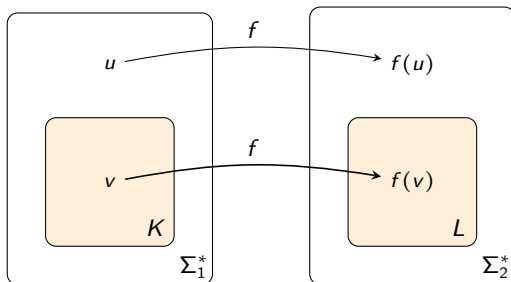
Reducciones: definición

Definición:

Sean lenguajes $K \subseteq \Sigma_1^*$ y $L \subseteq \Sigma_2^*$. Una **reducción** de K a L es una función computable $f : \Sigma_1^* \rightarrow \Sigma_2^*$ tal que para toda palabra $w \in \Sigma_1^*$:

$$w \in K \iff f(w) \in L$$

Idea:



Reducciones: definición

Definición:

Sean lenguajes $K \subseteq \Sigma_1^*$ y $L \subseteq \Sigma_2^*$. Una **reducción** de K a L es una función computable $f : \Sigma_1^* \rightarrow \Sigma_2^*$ tal que para toda palabra $w \in \Sigma_1^*$:

$$w \in K \iff f(w) \in L$$

Comentarios:

- También se le llama **reducción mucho a uno** (**many-to-one reduction**).
- La función $f : \Sigma_1^* \rightarrow \Sigma_2^*$ **no** tiene que ser biyectiva (ni inyectiva, ni sobre).
- Si existe una **reducción** de K a L , decimos que K se **reduce** a L y escribimos:

$$K \leq_m L.$$

- La relación $\leq_m$ entre lenguajes es **transitiva**:

$$\text{Si } K \leq_m L \text{ y } L \leq_m J, \text{ entonces } K \leq_m J.$$

(¿por qué?)

Reducciones: propiedades

Proposición:

Si $K \leq_m L$ y L es **decidible**, entonces K es **decidible**.

Demostración:

Sea f la reducción de K a L y M_L la MT que decide L .

La siguiente MT M_K decide K .

Sobre entrada w :

1. Computar $f(w)$.
2. Ejecutar M_L sobre $f(w)$.
3. Si M_L **acepta**, **aceptar**.
4. Si M_L **rechaza**, **rechazar**.

Tenemos que:

$$w \in K \iff f(w) \in L \iff M_L \text{ acepta } f(w) \iff M_K \text{ acepta } w$$

Reducciones: propiedades

Proposición:

Si $K \leq_m L$ y L es **decidable**, entonces K es **decidable**.

Corolario:

Si $K \leq_m L$ y K es **indecidable**, entonces L es **indecidable**.

Para demostrar que un lenguaje L es **indecidable**,
basta demostrar que otro lenguaje **indecidable** se reduce a L .

Recordatorio:

$K \leq_m L =$ “ L es igual o más difícil que K ”.

Reducciones: propiedades

Proposición:

Si $K \leq_m L$ y L es **recursivamente enumerable**, entonces K es **recursivamente enumerable**.

Demostración:

Sea f la reducción de K a L y M_L la MT que acepta L .

La siguiente MT M_K acepta K .

Sobre entrada w :

1. Computar $f(w)$.
2. Ejecutar M_L sobre $f(w)$.
3. Si M_L **acepta**, **aceptar**.
4. Si M_L **rechaza**, **rechazar**.

Tenemos que:

$$w \in K \iff f(w) \in L \iff M_L \text{ acepta } f(w) \iff M_K \text{ acepta } w$$

Reducciones: propiedades

Proposición:

Si $K \leq_m L$ y L es recursivamente enumerable, entonces K es recursivamente enumerable.

Corolario:

Si $K \leq_m L$ y K no es recursivamente enumerable, entonces L no es recursivamente enumerable.

Para demostrar que un lenguaje L no es recursivamente enumerable, basta demostrar que otro lenguaje no recursivamente enumerable se reduce a L .

Reducciones: ejemplos

Veamos que $\text{Accept} \leq_m \text{Halting}$.

Podemos tomar la siguiente reducción $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$:

$$f(\langle M, w \rangle) = \langle S_M, w \rangle$$

f es reducción:

- f es computable.
- $\langle M, w \rangle \in \text{Accept} \iff \langle S_M, w \rangle \in \text{Halting}$.

OJO:

En la definición de f , podemos ignorar las entradas w que **no** tienen la forma $\langle M, w \rangle$.

- Se asume que en ese caso, $f(w)$ se define como una palabra que **no** está en Halting.
(por ejemplo, $f(w) = \varepsilon$.)

Reducciones: ejemplos

Veamos que $\text{Accept} \leq_m \text{Halting}$.

Podemos tomar la siguiente reducción $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$:

$$f(\langle M, w \rangle) = \langle S_M, w \rangle$$

f es reducción:

- f es computable.
- $\langle M, w \rangle \in \text{Accept} \iff \langle S_M, w \rangle \in \text{Halting}$.

Como Accept es **indecidable**, obtenemos que Halting es **indecidable**.

Reducciones: ejemplos

Veamos que $\text{Halting} \leq_m \text{Halting}_\epsilon$.

Podemos tomar la siguiente reducción $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$:

$$f(\langle M, w \rangle) = \langle B_{M,w} \rangle$$

f es reducción:

- f es computable.
- $\langle M, w \rangle \in \text{Halting} \iff \langle B_{M,w} \rangle \in \text{Halting}_\epsilon$.

Como Halting es **indecidable**, obtenemos que Halting_ϵ es **indecidable**.

Ejercicios propuestos

- Para una palabra fija $u \in \{0,1\}^*$, defina el siguiente lenguaje:

$$\text{Halting}_u = \{\langle M \rangle \mid M \text{ es una MT y } M \text{ se detiene con } u\}.$$

Demuestre usando reducciones que Halting_u es indecidible.

- Para una palabra fija $u \in \{0,1\}^*$, defina el siguiente lenguaje:

$$\text{Accept}_u = \{\langle M \rangle \mid M \text{ es una MT y } M \text{ acepta } u\}.$$

Demuestre usando reducciones que Accept_u es indecidible.

Más problemas: Non-Empty

Definamos el siguiente lenguaje sobre $\{0, 1\}$:

$$\text{Non-Empty} = \{ \langle M \rangle \mid M \text{ es una MT y } L(M) \neq \emptyset \}.$$

Proposición:

Non-Empty es recursivamente enumerable.

Demostración: Propuesta.

Teorema:

Non-Empty es indecidible.

Corolario:

Non-Empty no es recursivamente enumerable.

Non-Empty $\notin R$

Demostraremos que $\text{Accept} \leq_m \text{Non-Empty}$. Esto implica $\text{Non-Empty} \notin R$.

Reducción f de Accept a Non-Empty:

$$f(\langle M, w \rangle) = \langle B_{M,w} \rangle$$

Pseudocódigo de $B_{M,w}$

Sobre entrada $x \in \{0,1\}^*$:

1. Borrar x y dejar w escrito en la cinta.
2. Ejecutar M sobre w . (y responder lo mismo que M .)

f es computable y se cumple que:

$$\langle M, w \rangle \in \text{Accept} \implies L(B_{M,w}) = \{0,1\}^* \implies \langle B_{M,w} \rangle \in \text{Non-Empty}.$$

$$\langle M, w \rangle \notin \text{Accept} \implies L(B_{M,w}) = \emptyset \implies \langle B_{M,w} \rangle \notin \text{Non-Empty}.$$

Es decir: $\langle M, w \rangle \in \text{Accept} \iff \langle B_{M,w} \rangle \in \text{Non-Empty}$.

Más problemas: All

Definamos el siguiente lenguaje sobre $\{0, 1\}$:

$$\text{All} = \{ \langle M \rangle \mid M \text{ es una MT y } L(M) = \{0, 1\}^* \}.$$

Teorema:

All no es recursivamente enumerable.

Teorema:

$\overline{\text{All}}$ no es recursivamente enumerable.

Corolario:

All es indecidible.

Paréntesis: reducciones y complementos

Proposición:

Si $K \leq_m L$, entonces $\overline{K} \leq_m \overline{L}$.

Demostración:

Si f es una reducción de K a L , también es reducción de $\overline{K}$ a $\overline{L}$.

$\overline{\text{All}} \notin \text{RE}$

La misma reducción anterior demuestra que $\text{Accept} \leq_m \text{All}$.

Esto implica que $\overline{\text{Accept}} \leq_m \overline{\text{All}}$ y entonces, $\overline{\text{All}} \notin \text{RE}$.

Reducción f de Accept a All :

$$f(\langle M, w \rangle) = \langle B_{M,w} \rangle$$

Pseudocódigo de $B_{M,w}$

Sobre entrada $x \in \{0, 1\}^*$:

1. Borrar x y dejar w escrito en la cinta.
2. Ejecutar M sobre w . (y responder lo mismo que M .)

f es computable y se cumple que:

$$\langle M, w \rangle \in \text{Accept} \implies L(B_{M,w}) = \{0, 1\}^* \implies \langle B_{M,w} \rangle \in \text{All}.$$

$$\langle M, w \rangle \notin \text{Accept} \implies L(B_{M,w}) = \emptyset \implies \langle B_{M,w} \rangle \notin \text{All}.$$

Es decir: $\langle M, w \rangle \in \text{Accept} \iff \langle B_{M,w} \rangle \in \text{All}$.

All $\notin$ RE

Demostraremos que $\overline{\text{Accept}} \leq_m \text{All}$. Esto implica All $\notin$ RE.

Reducción f de $\overline{\text{Accept}}$ a All:

$$f(\langle M, w \rangle) = \langle Z_{M,w} \rangle$$

Pseudocódigo de $Z_{M,w}$ (2 cintas)

Sobre entrada $x \in \{0,1\}^*$:

1. Escribir w en la cinta 2.
2. Ejecutar M sobre w en la cinta 2, una cantidad $|x|$ de pasos.
3. Si M **acepta**, **rechazar**.
4. Si M **no acepta**, **aceptar**.

f es computable y se cumple que:

$$\begin{aligned} \langle M, w \rangle \notin \text{Accept} &\implies L(Z_{M,w}) = \{0,1\}^* \implies \langle Z_{M,w} \rangle \in \text{All}. \\ \langle M, w \rangle \in \text{Accept} &\implies M \text{ acepta } w \text{ en } k \text{ pasos} \\ &\implies Z_{M,w} \text{ rechaza todo } x, \text{ con } |x| \geq k \implies L(Z_{M,w}) \neq \{0,1\}^* \implies \langle Z_{M,w} \rangle \notin \text{All}. \end{aligned}$$

Es decir: $\langle M, w \rangle \in \overline{\text{Accept}} \iff \langle Z_{M,w} \rangle \in \text{All}$.

Más problemas: Finite

Definamos el siguiente lenguaje sobre $\{0, 1\}$:

$$\text{Finite} = \{ \langle M \rangle \mid M \text{ es una MT y } L(M) \text{ es finito} \}.$$

Teorema:

Finite no es recursivamente enumerable.

Teorema:

Finite no es recursivamente enumerable.

Corolario:

Finite es indecidible.

$\overline{\text{Finite}} \notin \text{RE}$

La misma reducción anterior demuestra que $\text{Accept} \leq_m \text{Finite}$.
Esto implica que $\overline{\text{Accept}} \leq_m \overline{\text{Finite}}$ y entonces, $\overline{\text{Finite}} \notin \text{RE}$.

Reducción f de Accept a Finite :

$$f(\langle M, w \rangle) = \langle Z_{M,w} \rangle$$

Pseudocódigo de $Z_{M,w}$ (2 cintas)

Sobre entrada $x \in \{0, 1\}^*$:

1. Escribir w en la cinta 2.
2. Ejecutar M sobre w en la cinta 2, una cantidad $|x|$ de pasos.
3. Si M **acepta**, **rechazar**.
4. Si M **no acepta**, **aceptar**.

f es computable y se cumple que:

$$\langle M, w \rangle \notin \text{Accept} \implies L(Z_{M,w}) = \{0, 1\}^* \implies \langle Z_{M,w} \rangle \notin \text{Finite}.$$

$$\begin{aligned} \langle M, w \rangle \in \text{Accept} &\implies M \text{ acepta } w \text{ en } k \text{ pasos} \\ &\implies Z_{M,w} \text{ rechaza todo } x, \text{ con } |x| \geq k \implies L(Z_{M,w}) \text{ es finito} \implies \langle Z_{M,w} \rangle \in \text{Finite}. \end{aligned}$$

Es decir: $\langle M, w \rangle \in \text{Accept} \iff \langle Z_{M,w} \rangle \in \text{Finite}$.

Finite $\notin$ RE

Demostraremos que $\overline{\text{Accept}} \leq_m \text{Finite}$, usando una reducción que ya vimos.

Reducción f de $\overline{\text{Accept}}$ a Finite:

$$f(\langle M, w \rangle) = \langle B_{M,w} \rangle$$

Pseudocódigo de $B_{M,w}$

Sobre entrada $x \in \{0,1\}^*$:

1. Borrar x y dejar w escrito en la cinta.
2. Ejecutar M sobre w . (y responder lo mismo que M .)

f es computable y se cumple que:

$$\langle M, w \rangle \notin \text{Accept} \implies L(B_{M,w}) = \emptyset \implies \langle B_{M,w} \rangle \in \text{Finite}.$$

$$\langle M, w \rangle \in \text{Accept} \implies L(B_{M,w}) = \{0,1\}^* \implies \langle B_{M,w} \rangle \notin \text{Finite}.$$

Es decir: $\langle M, w \rangle \in \overline{\text{Accept}} \iff \langle B_{M,w} \rangle \in \text{Finite}.$